Real-Time Visual Servoing via Predictive Kalman Filtering

Andrii Bondarenko
ČVUT–FIT
bondaan3@fit.cvut.cz

May 11, 2026

1 Introduction

Direct visual tracking using an camera mounted on an open-loop, two-axis stepper motor gimbal introduces inherent feedback delays. Processing video frames at 30 FPS alongside neural network inference and mechanical inertia creates a system phase lag of approximately 100 ms.

Controllers reacting only to immediate visual inputs chase outdated target coordinates, causing continuous position overshoot and mechanical oscillation. This project implements a multi-threaded tracking platform that compensates for hardware latency by combining asynchronous perception pipelines with predictive kinematics.

2 Input Data & Preprocessing

The primary sensor feed is an RGB video stream captured via USB at 1280×720 resolution operating at 30 FPS. The physical tracking target (a hand) is positioned 1.0 to 1.5 meters from the lens.

To eliminate fisheye distortion and ensure linear step-to-pixel mapping across the canvas, incoming frames are un-distorted using an intrinsic camera matrix derived from offline chessboard calibration[1].

To prevent frame processing from blocking the user interface, the system delegates execution to asynchronous worker threads using Qt concurrency (‘QThread‘)[5]. Decoupling video capture, MediaPipe landmark extraction[4], and serial communication ensures consistent pipeline throughput, stabilizing end-to-loop processing latency to a mean of 19.7 ms.

3 Methodology & Algorithms

Extracted hand coordinates are processed by a discrete 2D Second-Order Kalman Filter[3] using a constant-acceleration kinematic model:

$$\mathbf{x} = [x, y, v_x, v_y, a_x, a_y]^T \tag{1}$$

To filter out single-frame MediaPipe tracking errors (hallucinations), updates are evaluated using Mahalanobis distance gating. Measurements exceeding the statistical threshold are discarded, allowing the state vector to update using only the internal transition matrix.

Filtered coordinates drive a discrete Proportional-Derivative (PD) controller mapping pixel errors to motor steps. Proportional gain (K_p) sets target pursuit speed, while derivative gain (K_d) damps mechanical momentum to prevent ringing.

An Ego-Motion Feedforward loop was tested to subtract commanded motor steps directly from the Kalman state vector. However, fluctuating USB transport latency caused desynchronization between state shifts and visual confirmation, inducing positive feedback instability. Consequently, open-loop feedforward compensation was disabled in favor of predictive phase-leading.

4 Results & Discussion

Tracking metrics were captured across continuous horizontal sweeps, vertical drops, and circular trajectories. Custom ONNX gesture classification models mapped an `Open` class to engage tracking loops and a `Closed` class to halt actuation.

System evaluation compares two configurations: a baseline reactive state (horizon = 0 ms, $K_d = 0$) and a tuned predictive state (horizon = 100 ms, $K_d = 0.01$). Performance is quantified by Tracking Error (distance from frame center to target) and Prediction Error (deviation of the kinematic model from actual target paths).

As shown in Table 1 and Figure 2, the baseline setup yields low prediction error (12.1 px) but high tracking error (108.6 px) because the motors continuously target outdated coordinates.

Setting the predictive horizon to 100 ms directly offsets the system phase lag. Illustrated by the pursuit arcs in Figure 1 and smaller error in Figure 3, this tuning reduces physical tracking error by 34% (to a mean of 71.6 px).

Table 1: Tracking Performance Metrics

Metric (Pixels)	Baseline	Tuned Lead
Prediction Horizon	0 ms	100 ms
Derivative Gain (K_d)	0.00	0.01
Mean Tracking Error	108.6	**71.6**
Median Tracking Error	69.4	**60.8**
Mean Prediction Error	**12.1**	44.7
Pred. Acc. (< 50 px)	99.7%	68.1%

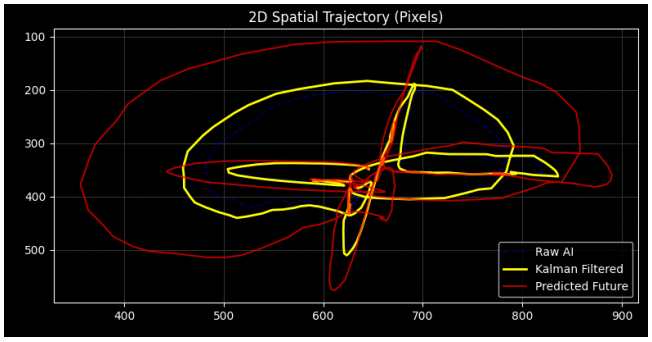


Figure 1: Tuned 2D Spatial Trajectory Pursuit

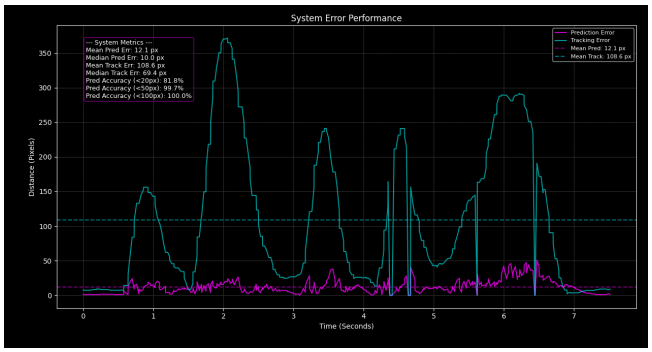


Figure 2: Baseline Error over Time (Horizon = 0)

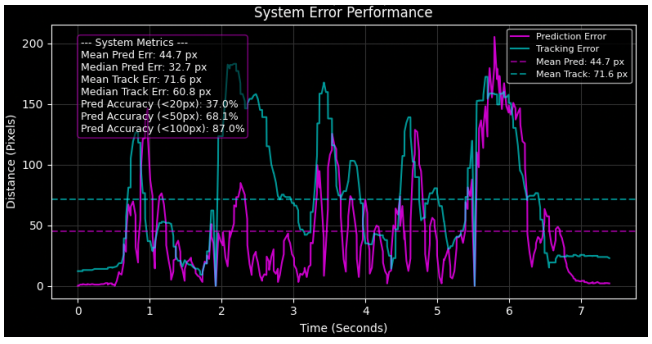


Figure 3: Tuned Error over Time (Horizon = 100 ms)

The kinematic prediction error inherently increases (12.1 px $\rightarrow$ 44.7 px) because circular and abrupt human hand trajectories deviate from linear constant-acceleration vectors over a 100 ms window. The platform trades mathematical model precision for improved hardware centering.

5 Conclusion

The system successfully maintains real-time target locking. Combining multithreaded perception with predictive Kalman filtering and derivative damping proves viable for tracking dynamic objects using delayed visual data.

Future implementation should replace open-loop step counting with absolute magnetic encoders mounted to the motor shafts to provide hardware-level state verification and improved RGB sensor to provide more stable and higher FPS video stream.

References

[1] Gary Bradski. The OpenCV Library. online, 2000. [cit. 2026-05-11] `https://github.com/opencv/opencv`.

[2] Google DeepMind. Gemini 3.1 Pro: Generative AI Assistance. online, 2026. [cit. 2026-05-11] Assisted with multithreaded phase synchronization logic, linter validation, and LaTeX structural layout. `https://gemini.google.com`.

[3] Roger R. Labbe. Kalman and Bayesian Filters in Python: Chapter 8.3.3. online, 2020. [cit. 2026-05-11] `https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python`.

[4] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, et al. MediaPipe: A Framework for Building Perception Pipelines. online, 2019. [cit. 2026-05-11] `https://arxiv.org/abs/1906.08172`.

[5] The Qt Company. PySide6: Qt for Python Documentation. online, 2026. [cit. 2026-05-11] `https://doc.qt.io/qtforpython-6/`.